Date: / / Page No.:

N	113
RESULT	114
TERM	116
ALPHA	104
ONE	115

Design Specification of assembler

(i) what is required in assembler
(i) identify the information necessary to perform a task
(ii) select a suitable D.S. to hold the info.
(iii) determine the processing necessary to maintain the info. in the data structure (DS)
(iv) Determine the processing necessary to perform the task

Adopt the above approach
There must be two phases :-
(1) Analysis (2) synthesis

(i) ⇒ Isolate the label, mnemonic opcode & operand field of a stt.
(ii) If a symbol is found in label field enter it in its symbol (LC) location counter into a table called TI table
(iii) Then check the validity of mnemonic opcode through a lookup in the mnemonic table

(1) Obtain the mnemonic code corresponding to the mnemonic from the MOT table
(ii) Obtain address of a memory operand from the symbol table
(iii) Synthesis the machine instruction or the M/C formed constraint

Date: / / Page No.:

(iv) perform LC processing.

Mnemonic Table :- Standard table with the reference by the analysis & synthesis phase while the symbolic table is constructed during the analysis phase & used during synthesis phase

Passes :- It means a complete scan of the src prog. exactly once

Passes structure of Assembler :-

1 passes	2 passes	multiple pass
If forward ref a prob while assering stt containing forward references To resolve it some extra data structure is required such as IIT (Intermediate	① No forward ref is there ② as LC processing an entry of symbols is done in first pass in second pass synthesis of target prog is done	

Some Advance Assembler Directory

(1) Origin (ORIGIN) < address specification >

capital likhna hai

Date: / / Page No.:

where addr. specificat" is an <operand specification> or <constant>

this directive indicates that LC processing should be reset to the addr. given by the addr. specification.

this stt is useful when the target prog doesnot consist of consecutive memory word.

Example :- ORIGIN 500 pass
OR
ORIGIN LAST+2

(2) EQU :- It define symbol to represent the <constant> or the <operand specification>

this differs from the DC or DS as it has no LC processing implied to it
Ex :- <symbol> EQU <operand specification>

The EQU stt only associate the name <symbol> with address specification

literal origin (3) LTORG :- This is used by the programmer to specify where literal should be placed their pool.

by default assembler places the literal after the end stt

Date: / / Page No.:

The assembler allocates a literal pool which contains all the literal used in the prog since the last LTORG stt.

All references to literals are forwd references by defination.

Ex:- LTORG
= '1'

Its important to seen in the computer using the base displacement mode of addressing
Ex :- computers of the IBM 360/370

Ex: Assembly lang

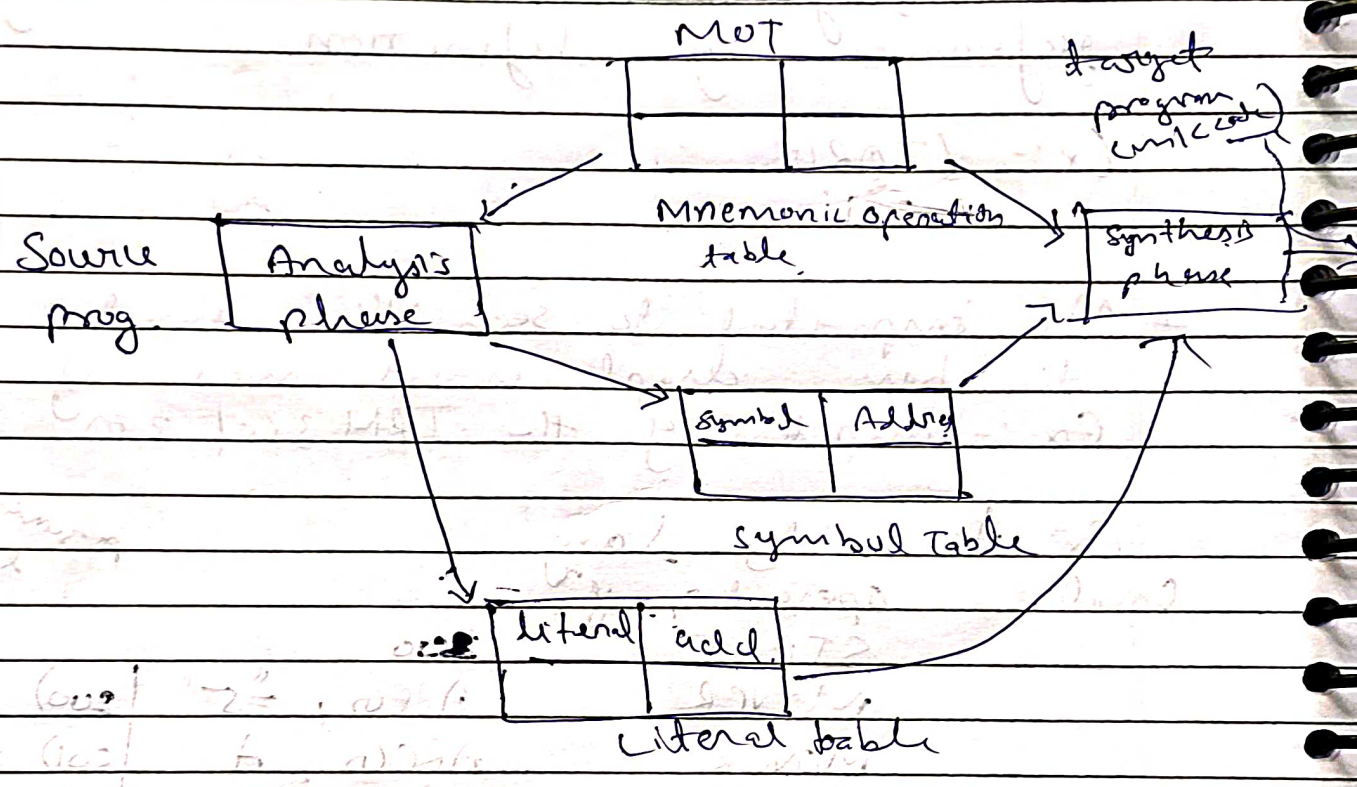
assuming stt length to be 1

Label	Operand specif^n		
	START	200	
	MOVER	AREG, ="5"	200) 04, 1, 206
	MOVER	AREG, A	201) 04, 1, 211
LOOP	~~LOOP~~ MOVER	AREG, A	202) 04, 1, 211
	MOVER	CREG, B	203) 04, 3, 212
	ADD	CREG, ="1"	204) 01, 3, 207
	BC	ANY, NEXT	205) 07, 6, —
iski LL processing nhi hoti ←	LTORG	~~ANY~~ ='5''	206) —
	5	='1'	207) —
NEXT	SUB	AREG, ~~=1~~ ='1'	208) 02, 01, 207
	BC	LT, BACK	209) 07, 1, —
LAST	STOP		210) 00
	ORIGIN	LOOP +2	
	MULT	CREG, B	211) 04, 03, 212

Date: / / Page No.:

	ORIGIN	LAST +1	
A	DS	1	211 , 207
BACK	~~BACK~~ EQU	LOOP	
B	DS	1	212
	END		

(*)



(*) DS of Assembler.

(1) OP Table or MOT + Pseudo OP table (POT) :-

Mnemonic operand corresponding m/c code
or

John J Donovan

Date: / / Page No.:

Assuming statement length to be 1 or 2.

Assembler of 360/370
Some more assembles directives or pseudo opcodes

(1) BALR (Branch & load register) :-

It is used to load the base register for Assembler for the next address in the secondary field ex: BALR 14, 0
(0-14 Register note)

(2) USING
It tell both to the Assembler that which register will be base register & what are its contents.

USING 20, 15 → Base register.
↓
content

(3) START
It tells the Assembler where the beginning of the program is & allows user to give a name to a program.

(4) END
It tells the assembler that this is the end of the program.
example:- END Address 4byte

Date: / / Page No.:

Assuming instruction length to 1st pass 360/370 program

```
JOHN    START   0
        USING   *,15
        L
        A
        ST
FOUR    DC
FIVE    DC
TEMP    DS
        END
```

2nd pass

```
0)   L   1   16(0,15)
4)   A   1   12(0,15)
8)   ST  1   20(0,15)
12)
16)
```

JOHN	0
FOUR	12
FIVE	16
TEMP	20